

Laporan Tugas Besar

Nama: Zaidan Kamil Munadi (103032430014)

Muhammad Cheng Ho Pulungan (103032400146)

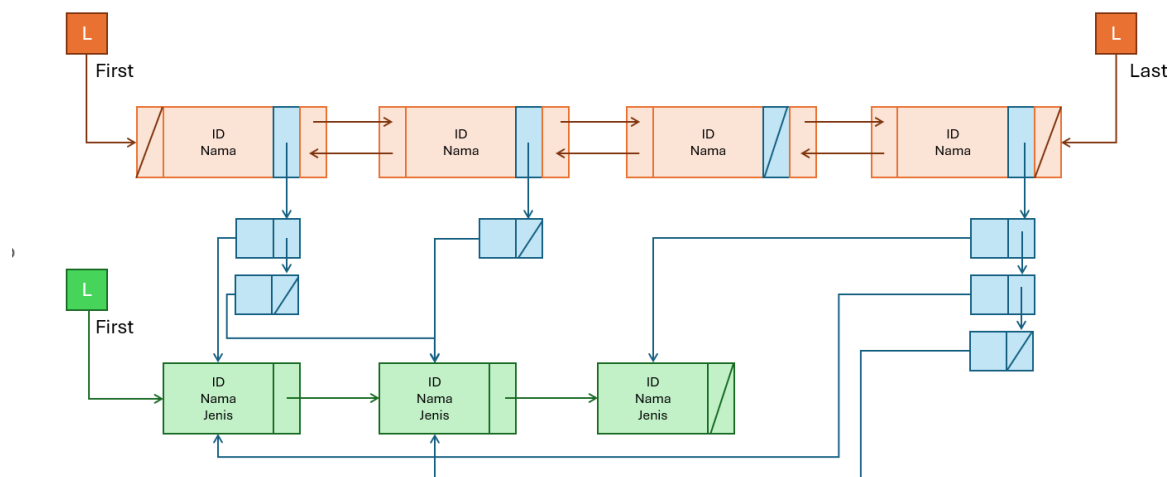
Judul: Manajemen Kepemilikan Data Hewan

Tipe MLL: A

Jenis List Parent: Doubly Linked List

Jenis List Child: Single Linked List

Model MLL:



Data Pemilik Hewan:

- ID (String)
- Nama Pemilik (String)

Data Hewan:

- ID Hewan (String)
- Nama Hewan (String)
- Jenis Hewan (String)

Spesifikasi Program

Program ini merupakan aplikasi berbasis console yang mengelola data Owner (Parent), data Pet (Child), serta Relasi kepemilikan antara Owner dan Pet.

Struktur data yang digunakan:

- **Owner** → Double Linked List (Memiliki pointer prev, next, first, last)
- **Pet** → Single Linked List (Memiliki pointer next, first)
- **Relation** → Single Linked List berisi pointer ke Owner & Pet (Memiliki pointer ownerPtr, petPtr, next, first)

Aplikasi mendukung berbagai operasi CRUD, pencarian relasi, statistik, dan pengurutan data.

Fitur-fitur (berdasarkan menu program)

1. Insert Owner (Parent) Menambahkan data pemilik hewan ke dalam ListOwner. Data yang dicatat: ID Owner, Nama Owner. Program mengecek apakah ID sudah ada, sehingga tidak terjadi duplikasi.

2. Insert Pet (Child) Menambahkan data hewan ke ListPet. Data hewan: ID Pet, Nama Pet, Jenis hewan. Program juga mengecek ID agar tidak terjadi duplikasi.

3. Connect (Hubungkan Owner & Pet) Menghubungkan seorang Owner dengan Pet tertentu. Setiap relasi memungkinkan:

- 1 Owner memiliki banyak hewan
- 1 hewan dapat dimiliki oleh lebih dari satu owner (shared ownership) Program memastikan relasi tidak duplikat sebelum dibuat.

4. Show All Owners Menampilkan seluruh data Owner dalam double linked list.

5. Show All Pets Menampilkan seluruh data hewan yang tersimpan dalam single linked list.

6. Show All Owners beserta Hewannya Menampilkan setiap Owner dan daftar hewan yang ia miliki.

7. Show All Pets beserta Pemiliknya Menampilkan setiap hewan dan siapa saja pemiliknya.

8. Cari Hewan milik Owner tertentu Input: ID Owner. Output: Daftar hewan yang dimiliki owner tersebut.

9. Cari Pemilik dari Hewan tertentu Input: ID Pet. Output: Daftar pemilik dari hewan tersebut.

10. Delete Relation (Putus Hubungan) Menghapus relasi antara Owner dan Pet tertentu tanpa menghapus data masternya.

11. Delete Owner (Cascade Delete) Menghapus Owner sekaligus menghapus semua relasi yang terhubung dengannya secara otomatis. ListOwner diperbarui dengan benar (penanganan hapus awal, akhir, atau tengah).

12. Delete Pet (Cascade Delete) Menghapus seekor hewan dan menghapus semua relasi yang berhubungan dengannya secara otomatis.

13. Statistik (Count) Menampilkan statistik berikut: a. Jumlah Hewan tak bertuan (orphan pets) b. Jumlah Owner tanpa hewan

14. Edit Relation (Pindah Tangan) Mengubah data relasi. Memungkinkan penggantian kepemilikan (ganti Owner) atau penggantian hewan yang dimiliki (ganti Pet) pada sebuah relasi yang sudah ada.

15. Edit Data Owner Mengubah nama Owner berdasarkan ID tertentu.

16. Edit Data Pet Mengubah nama dan jenis Pet berdasarkan ID tertentu.

17. Sort Owner by Name (Ascending) Mengurutkan data Owner berdasarkan nama secara alfabetis (A-Z) menggunakan algoritma Bubble Sort pada Double Linked List.

Persen kontribusi anggota dalam tim (total 100%)

Kontribusi Zaidan Kamil Munadi - 50%

1. **Mendesain struktur data:** a. Struct Owner, Pet, Relation b. Double Linked List Owner c. Single Linked List Pet & Relation
2. **Implementasi fungsi dasar:** a. createListOwner, createListPet, createListRelation b. insertOwner & insertPet c. Fungsi pencarian: findOwner, findPet, findRelation
3. **Menulis fungsi tampil data:** a. printOwners, printPets b. printAllOwnersWithPets
4. **Membuat menu utama & interface awal.**

Kontribusi Muhammad Cheng Ho Pulungan - 50%

1. **Implementasi fitur relasi:** a. connect (menghubungkan Owner-Pet) b. disconnect (hapus relasi)
2. **Mengimplementasikan fitur penghapusan & edit:** a. deleteOwner (cascade delete) b. deletePet (cascade delete) c. editRelation
3. **Menulis fungsi statistik & sorting:** a. countOrphanPets, countChildlessOwners b. sortOwnersByNama
4. **Testing keseluruhan program menggunakan data dummy & debugging.**

BUKTI Repsonsi Bersama Asprak



Lampiran hasil kode 80%

```
hewan.cpp x hewan.h x main.cpp x
1 | #include "hewan.h"
2 | void createListOwner(ListOwner &L) {
3 |     L.first = nullptr;
4 |     L.last = nullptr;
5 | }
6 |
7 | void createListPet(ListPet &L) {
8 |     L.first = nullptr;
9 | }
10 |
11 | void createListRelation(ListRelation &L) {
12 |     L.first = nullptr;
13 | }
14 |
15 | adrOwner alokasiOwner(string id, string nama) {
16 |     adrOwner P = new elmOwner;
17 |     P->info.id = id;
18 |     P->info.nama = nama;
19 |     P->next = nullptr;
20 |     P->prev = nullptr;
21 |     return P;
22 | }
23 |
24 | adrPet alokasiPet(string id, string nama, string jenis) {
25 |     adrPet P = new elmPet;
26 |     P->info.id = id;
27 |     P->info.nama = nama;
28 |     P->info.jenis = jenis;
29 |     P->next = nullptr;
30 |     return P;
31 | }
32 |
33 | adrRelation alokasiRelation(adrOwner P, adrPet C) {
34 |     adrRelation R = new elmRelation;
35 |     R->ownerPtr = P;
36 |     R->petPtr = C;
37 |     R->next = NULL;
38 |     return R;
39 | }
40 |
```

```

41 // ----- INSERT (A, B) -----
42 void insertOwner(ListOwner &L, adrOwner P) {
43     // Insert Last (Double Linked List)
44     if (L.first == nullptr) {
45         L.first = P;
46         L.last = P;
47     } else {
48         L.last->next = P;
49         P->prev = L.last;
50         L.last = P;
51     }
52 }
53
54 void insertPet(ListPet &L, adrPet C) {
55     // Insert Last (Single Linked List)
56     if (L.first == nullptr) {
57         L.first = C;
58     } else {
59         adrPet Q = L.first;
60         while (Q->next != nullptr) {
61             Q = Q->next;
62         }
63         Q->next = C;
64     }
65 }
66
67 // ----- FIND (G, H) -----
68 adrOwner findOwner(ListOwner L, string id) {
69     adrOwner P = L.first;
70     while (P != nullptr) {
71         if (P->info.id == id) return P;
72         P = P->next;
73     }
74     return nullptr;
75 }
76
77 adrPet findPet(ListPet L, string id) {
78     adrPet P = L.first;
79     while (P != nullptr) {
80         if (P->info.id == id) return P;
81         P = P->next;
82     }
83     return nullptr;
84 }
85

```

```

hewan.cpp x hewan.h x main.cpp x
76
77 adrPet findPet(ListPet L, string id) {
78     adrPet P = L.first;
79     while (P != nullptr) {
80         if (P->info.id == id) return P;
81         P = P->next;
82     }
83     return nullptr;
84 }
85
86 adrRelation findRelation(ListRelation L, string idOwner, string idPet) {
87     adrRelation R = L.first;
88     while (R != nullptr) {
89         if (R->ownerPtr->info.id == idOwner && R->petPtr->info.id == idPet) {
90             return R;
91         }
92         R = R->next;
93     }
94     return nullptr;
95 }
96
97 // ----- CONNECT / INSERT RELASI (C) -----
98 void connect(ListRelation &LR, ListOwner LP, ListPet LC, string idOwner, string idPet) {
99     adrOwner O = findOwner(LP, idOwner);
100     adrPet P = findPet(LC, idPet);
101
102     if (O != NULL && P != nullptr) {
103         // Cek apakah sudah ada relasi
104         if (findRelation(LR, idOwner, idPet) == nullptr) {
105             adrRelation R = alokasiRelation(O, P);
106             // Insert First pada Relasi (biar cepat)
107             R->next = LR.first;
108             LR.first = R;
109             cout << "Berhasil menghubungkan " << O->info.nama << " dengan " << P->info.nama << endl;
110         } else {
111             cout << "Relasi sudah ada." << endl;
112         }
113     } else {
114         cout << "Owner atau Pet tidak ditemukan." << endl;
115     }
116 }
117

```

```

119 void disconnect(ListRelation &LR, string idOwner, string idPet) {
120     // Menghapus node relasi tertentu
121     adrRelation P = LR.first;
122     adrRelation Prev = nullptr;
123     bool found = false;
124
125     while (P != nullptr && !found) {
126         if (P->ownerPtr->info.id == idOwner && P->petPtr->info.id == idPet) {
127             found = true;
128         } else {
129             Prev = P;
130             P = P->next;
131         }
132     }
133
134     if (found) {
135         if (Prev == nullptr) { // Hapus elemen pertama
136             LR.first = P->next;
137         } else {
138             Prev->next = P->next;
139         }
140         delete P;
141         cout << "Relasi dihapus." << endl;
142     } else {
143         cout << "Relasi tidak ditemukan." << endl;
144     }
145 }
146
147 void deleteOwner(ListOwner &LP, ListRelation &LR, string idOwner) {
148     // 1. Hapus semua relasi yang melibatkan owner ini dulu
149     adrRelation R = LR.first;
150     while (R != nullptr) {
151         adrRelation nextR = R->next;
152         if (R->ownerPtr->info.id == idOwner) {
153             disconnect(LR, idOwner, R->petPtr->info.id);
154         }
155         R = R->next;
156     }
157
158     // 2. Hapus Owner dari List Parent
159     adrOwner P = findOwner(LP, idOwner);
160     if (P != nullptr) {
161         if (P == LP.first) { // Hapus awal
162             LP.first = P->next;
163             if (LP.first != nullptr) {
164                 LP.first->prev = nullptr;
165             } else {
166                 LP.last = nullptr;
167             }
168         } else if (P == LP.last) { // Hapus akhir
169             LP.last = P->prev;
170             LP.last->next = nullptr;
171         } else { // Hapus tengah
172             P->prev->next = P->next;
173             P->next->prev = P->prev;
174         }
175         delete P;
176         cout << "Owner berhasil dihapus." << endl;
177     } else {
178         cout << "Owner tidak ditemukan." << endl;
179     }
180 }
181

```



```

181
182 void deletePet(ListPet &LC, ListRelation &LR, string idPet) {
183     // 1. Hapus semua relasi yang melibatkan pet ini
184     adrRelation R = LR.first;
185     while (R != nullptr) {
186         adrRelation nextR = R->next;
187         if (R->petPtr->info.id == idPet) {
188             disconnect(LR, R->ownerPtr->info.id, idPet);
189         }
190         R = R->next;
191     }
192
193     // 2. Hapus Pet dari List Child
194     adrPet P = findPet(LC, idPet);
195     if (P != nullptr) {
196         if (P == LC.first) {
197             LC.first = P->next;
198         } else {
199             adrPet Q = LC.first;
200             while (Q->next != P) {
201                 Q = Q->next;
202             }
203             Q->next = P->next;
204         }
205         delete P;
206         cout << "Pet berhasil dihapus." << endl;
207     } else {
208         cout << "Pet tidak ditemukan." << endl;
209     }
210 }
211
212 // ----- SHOW (J, K, L, M, N, O) -----
213 void printOwners(ListOwner L) {
214     adrOwner P = L.first;
215     cout << "=== List Owner ===" << endl;
216     while (P != nullptr) {
217         cout << "ID: " << P->info.id << " | Nama: " << P->info.nama << endl;
218         P = P->next;
219     }
220     cout << endl;
221 }
222
223 void printPets(ListPet L) {
224     adrPet P = L.first;
225     cout << "=== List Hewan ===" << endl;
226     while (P != nullptr) {
227         cout << "ID: " << P->info.id << " | Nama: " << P->info.nama << " (" << P->info.jenis << ")" << endl;
228         P = P->next;
229     }
230     cout << endl;
231 }
232
233 void printPetsByOwner(ListRelation LR, string idOwner) {
234     adrRelation R = LR.first;
235     bool found = false;
236     cout << "Hewan milik Owner ID " << idOwner << ":" << endl;
237     while (R != nullptr) {
238         if (R->ownerPtr->info.id == idOwner) {
239             cout << "- " << R->petPtr->info.nama << " (" << R->petPtr->info.jenis << ")" << endl;
240             found = true;
241         }
242         R = R->next;
243     }
244     if (!found) cout << "(Tidak ada hewan)" << endl;
245 }
246

```

```

260
261 void printAllOwnersWithPets(ListOwner LP, ListRelation LR) {
262     adrOwner P = LP.first;
263     while (P != nullptr) {
264         cout << "Owner: " << P->info.nama << endl;
265         printPetsByOwner(LR, P->info.id);
266         cout << "-----" << endl;
267         P = P->next;
268     }
269 }
270
271 void printAllPetsWithOwners(ListPet LC, ListRelation LR) {
272     adrPet P = LC.first;
273     while (P != nullptr) {
274         cout << "Hewan: " << P->info.nama << endl;
275         printOwnersByPet(LR, P->info.id);
276         cout << "-----" << endl;
277         P = P->next;
278     }
279 }
280
281 // ----- COUNT (P, Q, R, S) -----
282 int countPetsOfOwner(ListRelation LR, string idOwner) {
283     int count = 0;
284     adrRelation R = LR.first;
285     while (R != nullptr) {
286         if (R->ownerPtr->info.id == idOwner) count++;
287         R = R->next;
288     }
289     return count;
290 }
291
292 int countOwnersOfPet(ListRelation LR, string idPet) {
293     int count = 0;
294     adrRelation R = LR.first;
295     while (R != nullptr) {
296         if (R->petPtr->info.id == idPet) count++;
297         R = R->next;
298     }
299     return count;
300 }
301
302 int countOrphanPets(ListPet LC, ListRelation LR) {
303     int count = 0;
304     adrPet P = LC.first;
305     while (P != nullptr) {
306         if (countOwnersOfPet(LR, P->info.id) == 0) {
307             count++;
308         }
309         P = P->next;
310     }
311     return count;
312 }
313
314 int countChildlessOwners(ListOwner LP, ListRelation LR) {
315     int count = 0;
316     adrOwner P = LP.first;
317     while (P != nullptr) {
318         if (countPetsOfOwner(LR, P->info.id) == 0) {
319             count++;
320         }
321         P = P->next;
322     }
323     return count;
324 }

```

```

326 // ----- EDIT (T) -----
327 void editRelation(ListRelation &LR, ListOwner LP, ListPet LC, string oldOwnerID, string oldPetID, string newOwnerID, string newPetID) {
328     // Cari relasi lama
329     adrRelation R = findRelation(LR, oldOwnerID, oldPetID);
330     if (R == nullptr) {
331         cout << "Relasi lama tidak ditemukan!" << endl;
332         return;
333     }
334
335     // Cari Owner dan Pet baru
336     adrOwner newOwner = findOwner(LP, newOwnerID);
337     adrPet newPet = findPet(LC, newPetID);
338
339     if (newOwner != nullptr && newPet != nullptr) {
340         // Cek apakah relasi baru sudah ada
341         if (findRelation(LR, newOwnerID, newPetID) == nullptr) {
342             R->ownerPtr = newOwner;
343             R->petPtr = newPet;
344             cout << "Relasi berhasil diubah." << endl;
345         } else {
346             cout << "Relasi tujuan sudah ada, edit dibatalkan." << endl;
347         }
348     } else {
349         cout << "Data Owner baru atau Pet baru tidak valid." << endl;
350     }
351 }
352
353 // ----- UPDATE DATA (U, V) -----
354 void updateOwner(ListOwner &L, string id, string newName) {
355     adrOwner P = findOwner(L, id);
356     if (P != nullptr) {
357         P->info.nama = newName;
358         cout << "Data Owner berhasil diupdate." << endl;
359     } else {
360         cout << "Owner tidak ditemukan." << endl;
361     }
362 }
363
364 void updatePet(ListPet &L, string id, string newName, string newJenis) {
365     adrPet P = findPet(L, id);
366     if (P != nullptr) {
367         P->info.nama = newName;
368         P->info.jenis = newJenis;
369         cout << "Data Pet berhasil diupdate." << endl;
370     } else {
371         cout << "Pet tidak ditemukan." << endl;
372     }
373 }
374

```

```

374
375 // ----- SORTING (W) -----
376 void sortOwnersByName(ListOwner &L) {
377     // Bubble Sort pada Double Linked List (Swap Info)
378     if (L.first == nullptr || L.first->next == nullptr) {
379         return; // 0 or 1 element
380     }
381
382     bool swapped;
383     adrOwner P;
384     adrOwner LastPtr = nullptr;
385
386     do {
387         swapped = false;
388         P = L.first;
389
390         while (P->next != LastPtr) {
391             if (P->info.nama > P->next->info.nama) {
392                 // Swap Data
393                 Owner temp = P->info;
394                 P->info = P->next->info;
395                 P->next->info = temp;
396                 swapped = true;
397             }
398             P = P->next;
399         }
400         LastPtr = P;
401     } while (swapped);
402     cout << "List Owner berhasil diurutkan berdasarkan Nama (A-Z)." << endl;
403 }
404

```